

Behavior cloning of a Model-Predictive Planner into a Lightweight Neural network Policy

Implementation Report

Ali Pourgheisari, Mehran Rajabi

09.07.2026

Abstract

We try to clone a computationally expensive model-predictive expert into a lightweight feed-forward policy to reach the assigned goal in *Reacher* task of the DeepMind Control environment. The expert combines a learned latent world model with a Cross-Entropy Method (CEM) planner and requires on the order of two minutes of online search per reach. We collected 2,000 rollouts from the planner that succeeded, changed them into clean *approach-only* demonstrations by truncating each trajectory after its closest approach to the goal and removing non-reaching episodes, and trained a 68k-parameter multilayer perceptron that takes the arm’s proprioceptive state and the target joint configuration as input and outputs joint torques directly. The behavior cloning policy reaches the target in 68% of episodes in a short time. We explain how we collected and then cleaned the data, describe the model and how it was trained, and look at why a small network still works well even though its action-prediction error is fairly large.

1 Introduction

Sampling-based model-predictive control (MPC) can solve continuous control tasks without needing task-specific reward shaping or policy learning: at every step, the planner tries many possible action sequences using a dynamics model, and then runs the first action from the best sequence. When the dynamics model is a learned latent world model, this gives a strong, general-purpose controller. The problem is the cost: each step needs hundreds of imagined rollouts, which are then refined over many iterations, so even a simple task takes about two minutes of real computation time. This makes the planner hard to use in practice.

This report tackles that cost through behavior cloning (BC) [1]: we treat the planner as an expert, record the states it visits and the actions it takes, and then train a simple yet fast policy that copies its behavior in one forward pass. We use the two-joint *Reacher* task from the DeepMind Control Suite [2], in its goal-conditioned `qpos_match` version, where the arm has to move its joints to reach a target. Specifically, we (i) build a data-collection procedure

that runs the world-model/CEM expert from sampled starting points to sampled goals, and records the state and the action taken; (ii) clean up the rollouts into demonstrations that match the task’s success criterion; (iii) train a small MLP using only the proprioceptive state; and (iv) test it in closed loop, where it reaches 68% success at a much lower cost than the expert.

2 Background and Problem Setting

Task. Reacher is a planar two-joint manipulator simulated in MuJoCo [3]. Its configuration is $q = (q_1, q_2) \in \mathbb{R}^2$ with velocities $\dot{q} \in \mathbb{R}^2$, and the agent applies a bounded torque $a \in [-1, 1]^2$ per step. In the `qpos_match` variant the environment receives a target $q^* \in \mathbb{R}^2$, and an episode is a *success* if the fingertip reaches it at *any* step, and then the simulator terminates. This “reached-at-any-step” criterion drives our design: the ideal behavior to imitate is the *approach*, and motion after contact doesn’t have any impact.

Expert. The expert merges a learned latent world model with a CEM planner [4], following the idea of world-model-based control [5]. At each step, CEM samples action sequences from a Gaussian distribution, scores them by imagining what would happen under the world model, then refits the Gaussian using the best-performing (top- k) samples, called elites, and repeats this process. It controls the environment in a receding-horizon way, meaning it re-plans at every step of the process. We use 300 samples, 30 elites, 30 iterations, a planning horizon of 5, and an action block of 5. The expert is accurate but slow — which is exactly why we try to clone its behavior.

3 Related Work

Behavior cloning treats control as a supervised learning problem: it learns a mapping from states to actions using expert demonstrations [1]. Its main difficulty is covariate shift: once a cloned policy drifts away from the states it was trained on, it faces inputs it has never seen, and mistakes build up over time. Ross et al. [6] describe this problem and propose an interactive data-collection method called DAgger; our choices for curating and evaluating data take this failure mode into account. Cloning a world-model planner [5] into a reactive network gives up a small amount of final performance in exchange for a huge gain in inference speed.

A trickier issue is that the expert is a *stochastic optimizer*: for a given state, its action comes from a sample of an iterative search process, so the conditional distribution $p(a \mid \text{state})$ is spread out and can have multiple peaks. Regression losses that assume a single best output only capture its conditional mean — this is why distributional policy classes such as Diffusion Policy [7] exist. We keep the policy class simple here and come back to this point in Section 5.

4 Method

The pipeline has four stages: sampling reaching problems from a source dataset, generating expert demonstrations, cleaning them, and training the policy.

4.1 Reaching problems from a source dataset

We don’t learn from an existing policy — we just use a large recorded dataset as a bank of physically realistic states. The source dataset (`reacher_random`) has 10,000 episodes of 200 steps each. For each demonstration, we pick a valid starting step s and set the goal as the joint configuration $g = 25$ steps later in the same trajectory, $q^\star = q_{s+g}$ (this is valid only if $s + g + 1$ is actually in the episode). This makes sure every goal can eventually be reached, and gives us a nearly unlimited number of well-defined reaching problems.

4.2 Expert demonstration collection

For each rollout, we start the environment using two functions: one sets the state $(q_s, \dot{q}_s)$, and the other sets the target $q^\star$. Then we run the world-model/CEM policy for $T = 50$ steps. At each step t , we record the state $(q_t, \dot{q}_t)$ and the executed action a_t (clipped to $[-1, 1]$, as the environment applies it), and we store $q^\star$ once per episode. We collect $E = 2,000$ rollouts, each creating a new environment and a planner. Algorithm 1 summarizes the collection process and the curation that comes after it.

4.3 Feature representation

The policy input is the concatenation of the live internal state and the goal,

$$\phi_t = [q_t, \dot{q}_t, q^\star] \in \mathbb{R}^6, \quad (1)$$

and the output is the torque $a_t \in [-1, 1]^2$. Two choices matter. First, the goal $q^\star$ is *explicitly* in the input; without it the task is not identifiable, because the same arm state maps to different optimal actions under different goals. Second, we use the internal state details rather than pixels: $(q, \dot{q})$ is a sufficient statistic for this controller, whereas a rendered frame is $\approx 1.5 \times 10^5$ values, more cost to process, and does not even contain the goal. State features are lighter and more directly conditioned on the goal.

4.4 Demonstration curation

A raw rollout usually reaches the target somewhere near the middle of the episode and then moves away, so cloning the full episode would teach the policy to wander after arriving. Therefore we cut off each episode at its closest approach: using the shortest-arc angular error, we compute $t^\star = \arg \min_t \max_j |\text{wrap}(q_{t,j} - q_j^\star)|$ and keep only steps $0 \dots t^\star$. Then a filter removes episodes whose best angular error is larger than $\tau = 10^\circ$. This matches the ”reached-at-any-step” criterion (Section 2) and gives us only successful episodes: we keep 1,993 of 2,000 episodes (median closest-approach step 25 of 50), giving 58,715 (ϕ, a) pairs.

Algorithm 1 Demonstration collection and curation

```

1: Input: dataset  $\mathcal{D}$ , offset  $g=25$ , horizon  $T=50$ , tolerance  $\tau=10^\circ$ ;  $\mathcal{P} \leftarrow \emptyset$ 
2: for each demonstration  $e = 1 \dots E$  do
3:   sample valid start  $s$ ; set  $q^* \leftarrow q_{s+g}$ ; reset env to  $(q_s, \dot{q}_s)$  and target  $q^*$ 
4:   for  $t = 0 \dots T - 1$  do
5:      $a_t \leftarrow \text{clip}(\text{CEM-plan}(q_t, \dot{q}_t, q^*), -1, 1)$ ; record  $(q_t, \dot{q}_t, a_t)$ ; step env
6:   end for
7:    $t^* \leftarrow \arg \min_t \max_j |\text{wrap}(q_{t,j} - q_j^*)|$  {closest approach}
8:   if  $\max_j |\text{wrap}(q_{t^*,j} - q_j^*)| < \tau$  then
9:     add  $\{([q_t, \dot{q}_t, q^*], a_t)\}_{t=0}^{t^*}$  to  $\mathcal{P}$  {keep approach only}
10:  end if
11: end for
12: return  $\mathcal{P}$ 
    
```

4.5 Policy architecture and training

The policy is a multilayer perceptron with two ReLU hidden layers of width 256 and a final tanh to respect the torque bounds, $\pi_\theta(\phi) = \tanh(W_3 \sigma(W_2 \sigma(W_1 \phi + b_1) + b_2) + b_3)$, totalling $\approx 68\text{k}$ parameters. Inputs are standardised with training-split statistics that are stored alongside the weights so inference applies identical preprocessing. We minimise the mean-squared error to the expert action with Adam (lr 10^{-3} , weight decay 10^{-5}), batch size 256, for up to 300 epochs with early stopping (patience 15) on validation. The split is at the *episode* level (85%/15%), so no state from a validation episode appears in training. Table 1 lists the configuration.

Table 1: Configuration (left) and closed-loop evaluation of the distilled policy (right).

Component	Setting		
Source dataset	$10,000 \times 200$	Metric	Value
Collected demos	$2,000 \times 50$	Success (50 unseen)	68.0%
Goal offset g	25 steps	Demos retained	1993/2000
Expert	world model + CEM	Training pairs	58,715
CEM samp./elite/iter	300/30/30	Val. action MSE	0.122
Horizon / action block	5 / 5	Mean-action baseline	0.142
Feature ϕ (Eq. 1)	$q, \dot{q}, q^*$ (6-D)	Params	$\sim 68\text{k}$
Action	torque tanh (2-D)	Eval wall-clock	$\sim 18\text{ s}$
Network	6-256-256-2		
Optimiser	Adam 10^{-3} , wd 10^{-5}		
Batch/epochs/patience	256/300/15		

5 Results and Analysis

Protocol. We test the policy in closed loop. Using a fixed seed, we sample 50 held-out start/goal pairs, set them up using the same functions as before, and run π_θ for 50 steps per episode. We count it as a success if the target is reached at any step. Setting the seed for both the sampling and the environment makes the evaluation deterministic and reproducible.

Result. The BC policy reaches the target in **68%** of the 50 held-out episodes (34/50), and the full evaluation — including video rendering — finishes in about 18 seconds (Table 1). Because a single reach, solved in isolation, needs two CEM solves (each covering a 25-step block, since the planning horizon of 5 times the action block of 5 spans 25 environment steps) at a mean of 63.2s per solve — about two minutes of latency per reach — the policy replaces that search with a forward pass that takes a fraction of a millisecond per step while keeping most of the task’s performance.

Why the small model is sufficient. The validation action MSE (0.122) is only slightly below the value if we predicted the mean (0.142): the regressor explains little of the raw action variance. This is expected and doesn’t contradict the closed-loop result. Because the expert is a stochastic optimizer (Section 3), even nearly identical states can lead to different actions, so $p(a \mid \phi)$ has high variance, and an MSE regressor only recovers its *conditional mean*. For reaching the goal, that mean still points to the correct *direction* of motion — which is what actually drives the fingertip to the goal. So action error per step is a poor way to predict success; the real metric that matters is the closed-loop rollout, and a distributional action head [7] would capture the multimodality that MSE averages away. The remaining 32% of failures are consistent with covariate shift: once the policy leaves the states it was trained on, it reaches situations that are rare or missing from the training data, and its errors build up there. Because we split the data by episode rather than by individual step over the whole dataset, this failure rate isn’t inflated by data leakage — so it’s reliable.

6 Conclusion

We cloned an expensive world-model/CEM planner into a 68k-parameter state-based MLP for goal-based Reacher. The real value is in the data, not the model: sampling well-defined reaching problems, running the planner to create demonstrations, and most importantly, improving the data by curating them into a clean, goal-directed dataset that matches the success criterion. The policy reaches unseen targets 68% of the time at almost no running time. Natural next steps include: scaling up the demonstrations by sampling multiple goals per source episode; adding Cartesian fingertip information to the goal; using a distributional action head [7] to handle the expert’s multimodality; and using DAgger-style data aggregation [6] to reduce covariate shift. The trained policy and the full collection, training, and evaluation code come with this report.

References

- [1] D. A. Pomerleau. ALVINN: An Autonomous Land Vehicle in a Neural Network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1988.
- [2] Y. Tassa et al. DeepMind Control Suite. *arXiv preprint arXiv:1801.00690*, 2018.
- [3] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A Physics Engine for Model-Based Control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [4] R. Y. Rubinstein. The Cross-Entropy Method for Combinatorial and Continuous Optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.
- [5] D. Ha and J. Schmidhuber. Recurrent World Models Facilitate Policy Evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [6] S. Ross, G. J. Gordon, and J. A. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [7] C. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Robotics: Science and Systems (RSS)*, 2023.